

Netpos Payment Library

Processing card transactions to NIBSS and Interswitch from your own card-reading stack, without the Netplus Contactless SDK.

ARTIFACT

NetposNibssClient

VERSION

1.0.23

PLATFORM

Android · minSdk 21

AUDIENCE

Partner engineering

Contents

1	What the payment library is	8	Data models
2	Integration boundary	9	Supplying card data from your own reader
3	Installation	10	PIN block construction
4	Host-app prerequisites	11	Worked example, end to end
5	Credentials Netplus must issue	12	Transaction routing
6	Lifecycle: initialise → transact → finish	13	Responses and error handling
7	API reference	14	Reliability: reversals and re-push

01 What the payment library is

Netplus ships two independent Android components. Most integrations take both, but they are separable, and a partner who already reads cards needs only the second one.

COMPONENT	ARTIFACT	RESPONSIBILITY
Contactless SDK not covered here	<code>contactless-sdk</code>	Drives the NFC/EMV kernel and external readers, runs the EMV transaction flow, renders the PIN pad, and returns card data. Pulls in the Dspread, tpKernel and EMV-standalone native stacks.
Payment library THIS DOCUMENT	<code>NetposNibssClient</code> <code>netplus-epms</code>	Terminal configuration and key exchange with NIBSS, ISO 8583 message assembly, the encrypted socket to the acquirer, Interswitch routing, transaction persistence, reversals and backend logging.

The two meet at exactly one place: a `CardData` object plus an encrypted PIN block. Anything that can produce those four fields — your own EMV kernel, a certified reader SDK, an MSR head — can drive the payment library. The Contactless SDK is one such producer, and it is optional.

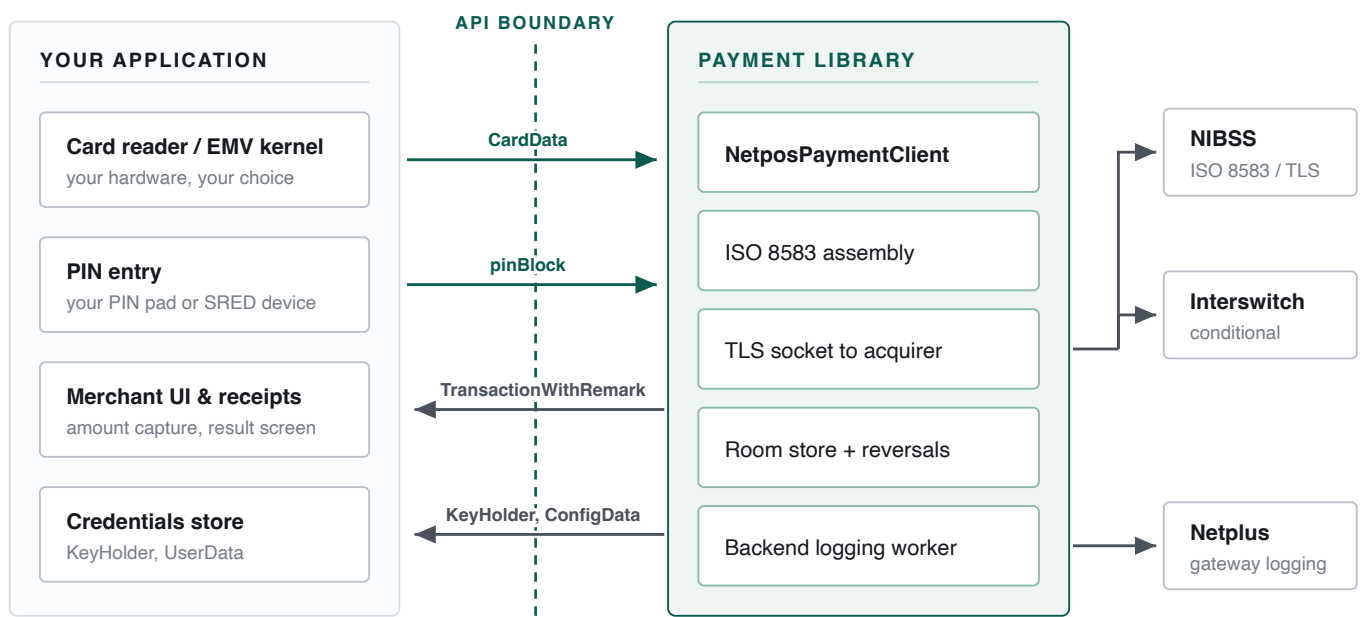
What you get

- **Terminal configuration.** Master/session/PIN key download from NIBSS, call-home, and daily key refresh, all handled by a single `init()` call.
- **Purchase processing.** Full ISO 8583 purchase over TLS to NIBSS, or through Interswitch when the terminal is provisioned for it.
- **Balance enquiry.** Account balance across the account types the issuer returns.
- **Durability.** Local Room persistence, automatic reversal on timeout, and a `WorkManager` job that re-pushes unlogged transactions to the Netplus backend.

What it does not do

- Read cards, run an EMV kernel, or manage reader hardware.
- Capture a PIN or render a PIN pad. You supply an already-encrypted PIN block.
- Print or render receipts.

02 Integration boundary



Your application owns everything left of the dashed line. The payment library never touches reader hardware; it receives card data and a PIN block and is responsible for everything from ISO 8583 assembly onward.

What you supply

Across the boundary you must provide exactly four things:

YOU SUPPLY	TYPE	SOURCE
UserData	JSON string	Netplus onboarding (§5)
CardData.track2Data	String (hex)	EMV tag 57 or MSR track 2
CardData.nibssIccSubset	String (hex TLV)	EMV tags from tag 55; empty for magstripe/track-2-only
CardData.pinBlock	String (hex)	ISO-0 block encrypted under the session PIN key

03 Installation

Repositories

Both artifacts are published through JitPack. Add the repository to `settings.gradle`:

```
// settings.gradle
dependencyResolutionManagement {
    repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
    repositories {
        google()
        mavenCentral()
        maven { url 'https://jitpack.io' }
    }
}
```

JITPACK AUTHENTICATION

The Netplus repositories are private. JitPack requires an auth token supplied as the Gradle `credentials { username ... }` block. Netplus issues this token during onboarding. Do not commit it; read it from `~/.gradle/gradle.properties` or CI secrets.

Dependencies

`NetposNibssClient` pulls `netplus-epms` transitively, but declare both so the EPMS entity classes (`CardData` , `KeyHolder` , `TransactionResponse`) resolve at compile time:

```
// app/build.gradle
dependencies {
    implementation 'com.github.netplusTeam:NetposNibssClient:1.0.23'
    implementation 'com.github.netplusTeam:netplus-epms:1.0.28'

    // Required at the call site – the API returns RxJava 2 Singles
    implementation 'io.reactivex.rxjava2:rxandroid:2.1.1'

    // Required – the library persists state through EasyPrefs (see §4)
    implementation 'com.pixplicity.easyprefs:EasyPrefs:1.10.0'

    // Note: contactless-sdk is deliberately absent.
}
```

Transitive footprint

Adding the payment library brings the following into your APK. Check these against what you already ship, and reconcile versions before you start:

LIBRARY	VERSION
androidx.room	2.2.6
androidx.work	2.7.1
androidx.paging	3.1.1
androidx.lifecycle	2.4.1
androidx.appcompat	1.2.0
rxjava2 / rxkotlin	2.2.21

LIBRARY	VERSION
retrofit2 + gson	2.9.0
okhttp logging	4.9.0
net.sf.j8583	1.17.0
logback-classic	1.2.3
firebase-messaging	bom 29.1.0
timber	4.7.1

TWO CONFLICTS TO EXPECT

- **SLF4J.** `logback-classic` collides with `slf4j-android` if anything else in your build supplies it. Exclude one: `configurations.configureEach { exclude group: 'org.slf4j', module: 'slf4j-android' }`
- **Firebase.** If you already use Firebase, the BOM at 29.1.0 will be aligned down unless you declare your own BOM after this dependency.

Native library and ABI

The AAR ships `libmodule-credentials.so`, which holds the encrypted acquirer endpoints and key components. It is provided for `arm64-v8a`, `armeabi-v7a`, `x86` and `x86_64`.

DO NOT FILTER ABIS AWAY

`UtilityParams` calls `System.loadLibrary("module-credentials")` in a static initialiser. If the ABI your device runs is stripped by an `abiFilters` block, the class fails to initialise and every call into the library throws `UnsatisfiedLinkError` at first use, not at install time. Ship all four ABIs, or filter to precisely the ones your terminal fleet runs.

Manifest permissions

The payment library itself needs only network access:

```
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
```

The Bluetooth and location permissions in the sample app belong to the Contactless SDK's external-reader discovery. Omit them unless your own reader stack requires them.

04 Host-app prerequisites

One piece of setup must happen before any library call. The rest is build configuration.

Initialise EasyPrefs

The library persists the key holder, terminal config data and user data through the `Prefs` singleton from EasyPrefs. That singleton has no context of its own. If your app does not build it, the first library call throws `RuntimeException: Prefs class not correctly instantiated`.

```
class App : Application() {
    override fun onCreate() {
        super.onCreate()
        Prefs.Builder()
            .setContext(this)
            .setMode(ContextWrapper.MODE_PRIVATE)
            .setPrefsName(packageName)
            .setUseDefaultSharedPreferences(true)
            .build()
    }
}
```

Register it in the manifest:

```
<application android:name=".App" ... />
```

Compile options

The AAR is built against Kotlin 1.9.22 with parcelize. Your module needs the `kotlin-parcelize` plugin only if you subclass the library's parcelable models; consuming them is fine without it. Java 8 desugaring or higher is required; Java 17 source/target is what the reference integration uses.

ProGuard / R8

Models cross the API boundary as Gson-serialised JSON, so field names must survive minification. If you enable R8 in release, keep the model packages:

```
-keep class com.netpluspay.nibssclient.models.** { *; }
-keep class com.netpluspay.nibssclient.work.** { *; }
-keep class com.danbamtale.epmslib.entities.** { *; }
-keepclassmembers class com.netpluspay.nibssclient.util.UtilityParams { native <methods>; }
```

Release build

The reference integration carries development settings that must not ship:

- The transitive `okhttp logging-interceptor` is set to `BODY` level, which writes full request and response bodies to logcat. Disable it in release.

- `usesCleartextTraffic="true"` is set in the sample manifest. Remove it.
- Set `FLAG_SECURE` on any activity that displays card data or a PIN pad.
- Keep PAN, track 2, PIN and key material out of every log statement.

THREADING

Every entry point returns an `RxJava 2 Single` and performs blocking socket I/O. Always `subscribeOn(Schedulers.io())` and `observeOn(AndroidSchedulers.mainThread())`. Nothing in the library switches threads for you.

05 Credentials Netplus must issue

These values populate `UserData`. Every field is a `String` and none may be null. Which of them are populated also determines transaction routing (§12).

FIELD	REQUIRED	MEANING
<code>terminalId</code>	REQUIRED	8-character NIBSS Terminal ID (TID). Drives key exchange and appears in ISO field 41. Wrong TID is the most common cause of a failed <code>init()</code> .
<code>partnerId</code>	REQUIRED	Netplus partner UUID. Namespaces cached tokens and identifies you to the Netplus gateway.
<code>businessName</code>	REQUIRED	Merchant trading name; used in backend logging.
<code>partnerName</code>	REQUIRED	Your organisation's name as registered with Netplus.
<code>businessAddress</code>	REQUIRED	Merchant location. Feeds the card-acceptor name/location element.
<code>customerName</code>	REQUIRED	Default cardholder label when the card supplies none.
<code>terminalSerialNumber</code>	CONDITIONAL	Hardware serial of the terminal. Required for Interswitch routing ; may be empty for NIBSS-only.
<code>mid</code>	CONDITIONAL	Merchant ID. Required for Interswitch routing ; may be empty for NIBSS-only.
<code>bankAccountNumber</code>	CONDITIONAL	Settlement account. Required for Interswitch routing ; may be empty for NIBSS-only.
<code>institutionalCode</code>	OPTIONAL	Institution code used in the Interswitch transfer payload.

ROUTING IS IMPLICIT

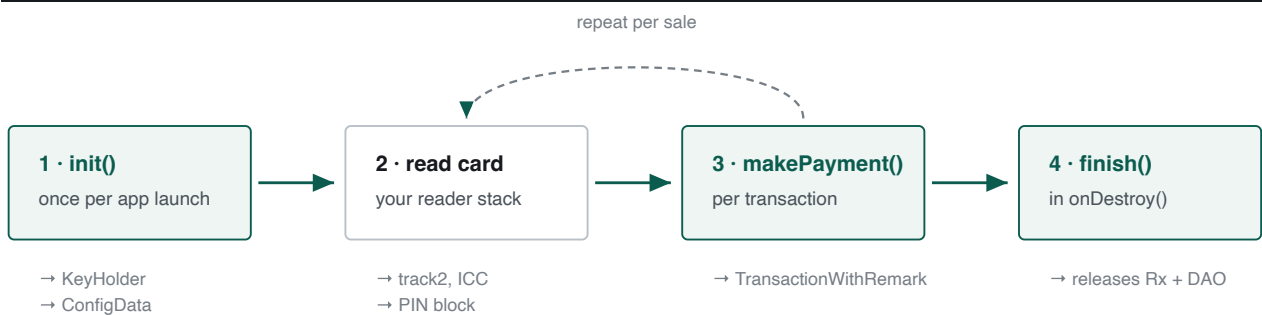
If `mid`, `bankAccountNumber` and `terminalSerialNumber` are all non-empty, every purchase and balance enquiry is routed through Interswitch. If any one is empty, it goes direct to NIBSS. There is no explicit routing flag, so populate these three deliberately rather than incidentally.

Beyond `UserData`, onboarding must also supply:

- The **JitPack access token** for the private Maven repository.
- Confirmation that your TID is **provisioned on the NIBSS host** and enabled for the transaction types you intend to run.

- A **test TID** for certification, if you need one distinct from production.

06 Lifecycle: initialise → transact → finish



`init()` is idempotent and safe to call on every screen entry; it decides internally whether a full configuration or a lightweight call-home is needed.

When keys refresh

`init()` inspects the timestamp of the last successful configuration and takes one of three paths:

CONDITION	ACTION
Last configuration was not today	Full terminal configuration: master key, session key, PIN key and parameter download. Returns a populated <code>Pair(KeyHolder, ConfigData)</code> .
Configured today, keys cached	Call-home only. Returns <code>Pair(null, null)</code> on success. The cached values in preferences remain valid.
Call-home fails	Falls back automatically to a full configuration.

THE NULL-PAIR TRAP

Because the call-home path returns `Pair(null, null)`, a naive `response.first!!` will crash on the second launch of the day. Always null-check, and treat a null pair as *success with keys already cached*. Read them back with your own persisted copy rather than overwriting with null.

07 API reference

The entire public surface is the Kotlin `object` `com.netpluspay.nibssclient.service.NetposPaymentClient`, a singleton, so there is nothing to construct. Five methods matter.

init

```
fun init(
    context: Context,
    serializedUserData: String
): Single<Pair<KeyHolder?, ConfigData?>>
```

Registers the terminal, performs key exchange with NIBSS, schedules the transaction re-push worker, and acquires an Interswitch token when the terminal is provisioned for it. Call this before anything else.

PARAMETER	NOTES
context	Any context; the application context is retained for WorkManager and Room.
serializedUserData	Gson JSON of a <code>UserData</code> instance, as a <i>string</i> rather than the object.

Returns a pair of the key holder and terminal config data, either or both of which may be null (see §6). **Errors** surface as `NibssClientException` carrying a `NibssError` with `possibleCause` and `possibleSolution` fields.

logUser

```
fun logUser(context: Context, serializedUserData: String)
```

Persists `UserData` without triggering configuration. `init()` calls this internally, so you rarely need it directly. The exception is switching merchant or terminal identity mid-session, where you should call `logUser()` and then `init()` again.

makePayment

```
fun makePayment(  
    context: Context,  
    terminalId: String? = "",  
    makePaymentParams: String,  
    cardScheme: String,  
    cardHolder: String,  
    rrn: String = "",  
    stan: String = ""  
): Single<TransactionWithRemark?>
```

PARAMETER	NOTES
terminalId	Overrides the TID from <code>UserData</code> . Pass the configured TID.
makePaymentParams	Gson JSON of a <code>MakePaymentParams</code> . Serialise <i>after</i> setting <code>cardData.pinBlock</code> .
cardScheme	Card brand as a display string: <code>"Verve"</code> , <code>"Mastercard"</code> , <code>"Visa"</code> . Determined by your kernel from the AID or PAN range.
cardHolder	Cardholder name. Use track 1 if available, otherwise a constant such as <code>"CUSTOMER"</code> .
rrn	Retrieval reference number. Leave empty to let the library obtain one from the Netplus RRN service.
stan	System trace audit number. Leave empty for library-generated.

TWO SILENT-NULL CONDITIONS

`makePayment` returns `Single.just(null)` , and does *not* error, when `MakePaymentParams.remark` is null or empty, or when the JSON fails to deserialise. Both surface only as a toast. Treat a null emission as a caller-side bug and assert on it.

balanceEnquiry

```
fun balanceEnquiry(  
    context: Context,  
    cardData: CardData,  
    accountType: String  
): Single<CheckAccountBalanceResponse>
```

Balance enquiry against the issuer. Unlike `makePayment` , this takes a `CardData` *object* rather than JSON, and the account type as the enum's `name` , for example `IsoAccountType.SAVINGS.name` .

A balance enquiry is a PIN-verified transaction: `cardData.pinBlock` must be set. Full ICC data is expected here, so populate `nibssIccSubset` .

finish

```
fun finish()
```

Clears the internal RxJava `CompositeDisposable` and releases the Room DAO. Call from `onDestroy()` of the activity or fragment that owns the payment flow. Because the client is a singleton, failing to call this leaks subscriptions across screens.

A note on the string parameters

`init`, `logUser` and `makePayment` take Gson-serialised JSON rather than typed parameters. The compiler will therefore not catch a malformed payload. Construct the typed model first and call `gson.toJson(it)` at the call site; do not hand-write the JSON.

08 Data models

DUPLICATE CLASS NAMES

CardData , IsoAccountType , TransactionType and KeyHolder each exist in *two* packages: com.danbamitale.epmslib.* and com.netpluspay.nibssclient.models.* . They are not interchangeable, and the IDE will happily import the wrong one. The table below states which package each API expects.

USED BY	TYPE	CORRECT PACKAGE
MakePaymentParams.cardData	CardData	com.danbamitale.epmslib.entities
balanceEnquiry(cardData)	CardData	com.danbamitale.epmslib.entities
MakePaymentParams.accountType	IsoAccountType	com.netpluspay.nibssclient.models
MakePaymentParams.transactionType	TransactionType	com.danbamitale.epmslib.entities
init() return value	KeyHolder	com.danbamitale.epmslib.entities

In practice the enums serialise by name, so a mismatched IsoAccountType is survivable across the JSON boundary. A mismatched CardData is not; it fails to compile, which is the better outcome.

UserData

```
data class UserData(  
    var businessName: String,  
    var partnerName: String,  
    var partnerId: String,  
    var terminalId: String,  
    var terminalSerialNumber: String,  
    var businessAddress: String,  
    var customerName: String,  
    val mid: String,  
    val institutionalCode: String,  
    val bankAccountNumber: String  
)
```

Positional order matters when constructing. See §5 for field semantics.

CardData — com.danbamitale.epmslib.entities

```
data class CardData(  
    val track2Data: String,           // EMV tag 57, hex, 'D' separator  
    val nibssIccSubset: String,       // NIBSS TLV subset; "" for track-2-only  
    val panSequenceNumber: String,    // tag 5F34, left-padded to 3 digits  
    val posEntryMode: String = "051"  
) {  
    var pinBlock: String? = null      // set this – see §10  
  
    // derived automatically from track2Data:  
    val pan, serviceCode, expiryDate, acquiringInstitutionIdCode  
}
```

MakePaymentParams

```
data class MakePaymentParams(  
    var action: String = "makePayment",  
    var terminalId: String? = ...,  
    var amount: Long = 0, // KOB0, minimum 200  
    var otherAmount: Long = 0, // cashback, in kobo  
    var transactionType: TransactionType = TransactionType.PURCHASE,  
    var accountType: IsoAccountType = IsoAccountType.DEFAULT_UNSPECIFIED,  
    var cardData: CardData,  
    var remark: String? = null // MUST be non-empty  
)
```

AMOUNTS ARE IN KOB0

`amount` is a minor-unit `Long`. ₦100.00 is `10000`. The host rejects anything below **200 kobo** with "Amount should not be less than 200 kobo". Never pass a naira value.

IsoAccountType — com.netpluspay.nibssclient.models

CONSTANT	ISO CODE	USE
DEFAULT_UNSPECIFIED	00	Let the issuer choose
SAVINGS	10	Savings account
CURRENT	20	Current account
CREDIT	30	Credit account
UNIVERSAL_ACCOUNT	40	Universal
INVESTMENT_ACCOUNT	50	Investment
BONUS_ACCOUNT	91	Bonus / loyalty

TransactionWithRemark — returned by makePayment

The fields you will actually read:

FIELD	TYPE	MEANING
responseCode	String	"00" means approved. Anything else is a decline (§13).
responseMessage	String	Human-readable outcome from the host.
rrn	String	Retrieval reference number. Your primary reconciliation key.
STAN	String	System trace audit number.
authCode	String	Issuer authorisation code. Print on the receipt.
amount	Int	Authorised amount, in kobo.
maskedPan	String	PAN safe to display and print.
cardLabel	String	Application label from the card.

FIELD	TYPE	MEANING
cardHolder	String	Cardholder name as sent.
cardExpiry	String	YYMM .
AID	String	EMV application identifier. Required on the receipt.
TVR / TSI	String	Terminal verification results and status. Required on the receipt.
appCryptogram	String	ARQC/TC from the card.
responseDE55	String	Issuer ICC response, TLV. Feed back to your kernel for second-generate AC and issuer scripts.
terminalId merchantId	String	As transmitted; echo on the receipt.
transmissionDateTime localDate_13 / localTime_12	String	Host and local timestamps.

CheckAccountBalanceResponse

```
data class CheckAccountBalanceResponse(
    val responseCode: String,
    val responseMessage: String,
    val accountBalances: List<AccountBalanceResponse> // accountType + amount (kobo)
)
```


09 Supplying card data from your own reader

Your kernel must produce four values. Everything else the library derives or supplies itself.

track2Data REQUIRED

The track 2 equivalent data: EMV **tag 57** for chip and contactless, or the physical track 2 for magstripe. Supply it as an uppercase hex string with the field separator as `D`, no sentinels, no LRC:

```
5061234567890123D2512601000000000000F
└── PAN ─┘ sep └YMM┘ └svc┘ └ discretionary ┘
```

The library parses PAN, expiry date, service code and acquiring institution code out of this string. A malformed separator produces a `StringIndexOutOfBoundsException` at construction, not at transaction time.

nibssIccSubset CONDITIONAL

NIBSS accepts only a specific subset of EMV tags in field 55, in a specific order. If your kernel emits the full tag-55 blob, pass it through the library's helper rather than assembling the subset by hand:

```
val icc = CardData.getNibssTags(fullIccHexFromKernel)
```

The tags retained, in the order NIBSS expects:

```
9F26 9F27 9F10 9F37 9F36 95 9A 9C 9F02 5F2A 82
9F1A 9F34 9F33 9F35 9F1E 84 9F09 9F03 5F34 8E
```

For a track-2-only flow — magstripe, or a contactless tap that produces no ICC data — pass an **empty string**. For a chip transaction this field is mandatory; an empty subset on a chip card will be declined.

BUILDING CARDDATA STRAIGHT FROM TAG 55

If your kernel returns one contiguous TLV blob, `CardData.initCardDataFromTrack(fullIcc)` extracts track 2 (tag 57), the NIBSS subset, PAN sequence (5F34) and POS entry mode (9F39) in a single call. You then only need to set `pinBlock`.

panSequenceNumber REQUIRED

EMV **tag 5F34**, left-padded with zeros to three digits: `"001"`, not `"1"`. Where the card supplies none, use `"000"`.

posEntryMode REQUIRED

EMV **tag 9F39**, or a literal appropriate to how the card was read. This tells the issuer how to interpret the CVM and liability:

VALUE	ENTRY METHOD
"051"	Chip (contact), PIN-capable terminal (library default)
"071"	Contactless chip (EMV mode)
"021"	Magnetic stripe read, full track
"011"	Manual PAN entry
"911"	Contactless magstripe (MSD)

The value must match how the card was actually read. Declaring a chip read as magstripe shifts chargeback liability to the acquirer.

pinBlock REQUIRED

See §10. This is the field that most often goes wrong.

10 PIN block construction

The library expects an **ISO 9564 format 0** PIN block, encrypted under the clear session PIN key that `init()` returned. It will not build one for you, and it will not validate one before transmitting it.

Obtaining the PIN key

`init()` returns a `KeyHolder` holding the encrypted master, session and PIN keys. The clear PIN key is exposed as an extension property that decrypts on access:

```
import com.danbamitale.epmslib.entities.clearPinKey

val pinKey: String = keyHolder.clearPinKey
```

Persist the `KeyHolder` — not the derived clear key — between sessions, and re-derive `clearPinKey` on each use.

If your PIN pad returns a clear PIN

Use the helper in `netplus-epms`. It performs the full ISO-0 construction and 3DES encryption:

```
import com.danbamitale.epmslib.comms.NewPinBlockCalculation

val pinBlock = NewPinBlockCalculation.getEncryptedPinBlock(
    pin      = enteredPin,           // 4–6 digits, clear
    cardPan  = cardData.pan,        // full PAN from track 2
    pinKey   = keyHolder.clearPinKey
)
cardData.pinBlock = pinBlock
```

What that helper does

1. **Block 1** — `"0"` + PIN length + the PIN, right-padded with `F` to 16 hex characters. A 4-digit PIN `1234` becomes `041234FFFFFFFF`.
2. **Block 2** — `"0000"` + PAN characters 4 through 15 (the twelve rightmost digits excluding the check digit).
3. **XOR** the two blocks.
4. **3DES-encrypt** the result under the clear PIN key.

IF YOUR PIN PAD ENCRYPTS IN HARDWARE

Do not use the helper; it needs a clear PIN, which a hardware pad will not give you. Arrange with Netplus for your pad's key to be injected under the same key hierarchy, so its output block is directly acceptable, then assign that block to `cardData.pinBlock` unchanged.

PIN-less transactions

Where the CVM resolved to no-CVM — a low-value contactless tap, or signature — leave `pinBlock` as `null`. Do not send an empty string; the library distinguishes null from empty when building field 52.

ORDERING

`MakePaymentParams` is serialised to JSON at the call site. Set `cardData.pinBlock` *before* calling `gson.toJson(params)` , or the PIN block will be silently absent from the payload and the transaction will be declined for an invalid PIN.

11 Worked example, end to end

A complete purchase with no Contactless SDK present. `YourCardReader` stands in for your own kernel.

1 – Application setup

```
class App : Application() {
    override fun onCreate() {
        super.onCreate()
        Prefs.Builder()
            .setContext(this)
            .setMode(ContextWrapper.MODE_PRIVATE)
            .setPrefsName(packageName)
            .setUseDefaultSharedPreferences(true)
            .build()
    }
}
```

2 – Configure the terminal

```
private val gson = Gson()
private val disposables = CompositeDisposable()
private var keyHolder: KeyHolder? = null

private val userData = UserData(
    businessName      = "Acme Stores",
    partnerName       = "Acme",
    partnerId         = BuildConfig.NETPLUS_PARTNER_ID,
    terminalId        = BuildConfig.NETPLUS_TID,
    terminalSerialNumber = "", // empty → NIBSS route
    businessAddress   = "12 Marina, Lagos",
    customerName      = "CUSTOMER",
    mid               = "",
    institutionalCode  = "",
    bankAccountNumber = ""
)

private fun configureTerminal() {
    disposables.add(
        NetposPaymentClient.init(this, gson.toJson(userData))
            .subscribeOn(Schedulers.io())
            .observeOn(AndroidSchedulers.mainThread())
            .subscribe { response, error ->
                error?.let { showError("Terminal configuration failed: " + it.localizedMessage);
            }
    }

    // A same-day call-home returns Pair(null, null) – keys are already cached.
    response?.first?.let { fresh ->
        keyHolder = fresh
        Prefs.putString(KEY HOLDER, gson.toJson(fresh))
    }
    if (keyHolder == null) {
        keyHolder = gson.fromJson(Prefs.getString(KEY HOLDER), KeyHolder::class.java)
    }
    onTerminalReady()
}
```

```

    )
}

```

3 – Read the card with your own stack

```

private fun onTerminalReady() {
    val pinKey = keyHolder?.clearPinKey ?: return showError("Terminal not configured")

    YourCardReader.startTransaction(amountKobo = 10_000L) { read ->
        val cardData = CardData(
            track2Data      = read.track2,                // tag 57
            nibssIccSubset  = CardData.getNibssTags(read.fullIcc), // tag 55 subset
            panSequenceNumber = read.panSequence.padStart(3, '0'), // tag 5F34
            posEntryMode     = "051"                      // tag 9F39
        )

        cardData.pinBlock = NewPinBlockCalculation.getEncryptedPinBlock(
            pin = read.clearPin, cardPan = cardData.pan, pinKey = pinKey
        )

        makePayment(cardData, amountKobo = 10_000L, scheme = read.cardScheme)
    }
}

```

4 – Process the payment

```

private fun makePayment(cardData: CardData, amountKobo: Long, scheme: String) {
    val params = MakePaymentParams(
        amount      = amountKobo,                // kobo; ≥ 200
        terminalId   = userData.terminalId,
        cardData     = cardData,                // pinBlock already set
        transactionType = TransactionType.PURCHASE,
        accountType  = IsoAccountType.SAVINGS,
        remark       = "Order #4471"            // MUST be non-empty
    )

    disposables.add(
        NetposPaymentClient.makePayment(
            context      = this,
            terminalId    = userData.terminalId,
            makePaymentParams = gson.toJson(params), // serialise AFTER pinBlock is set
            cardScheme    = scheme,
            cardHolder     = "CUSTOMER"
        )
        .subscribeOn(Schedulers.io())
        .observeOn(AndroidSchedulers.mainThread())
        .subscribe(
            { txn ->
                if (txn == null) { showError("Invalid request – check remark and params");
                return@subscribe }
                when (txn.responseCode) {
                    "00" -> onApproved(rrn = txn.rrn, auth = txn.authCode, pan = txn.maskedPan)
                    else -> onDeclined(code = txn.responseCode, message = txn.responseMessage)
                }
            },
            { error -> onFailure(error) }
        )
    )
}

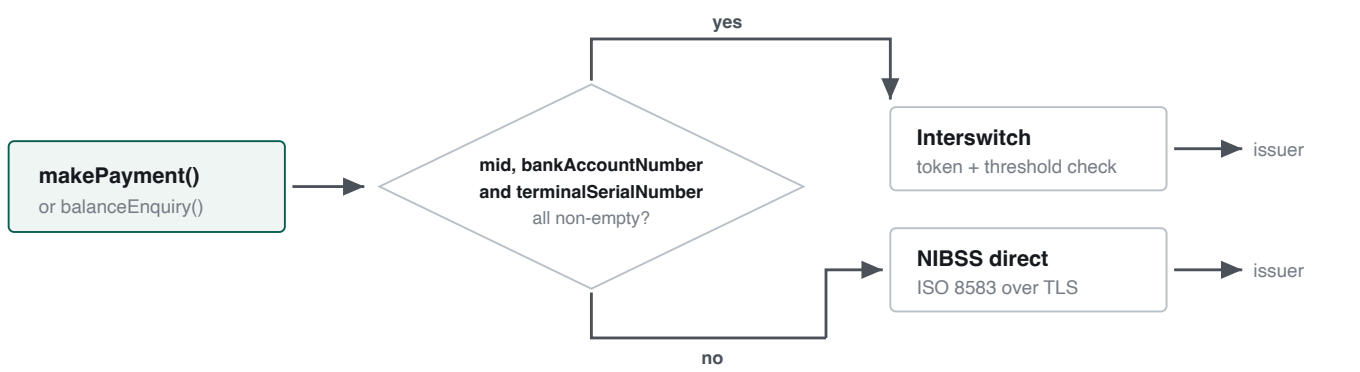
```

5 – Release

```
override fun onDestroy() {  
    super.onDestroy()  
    disposables.clear()  
    NetposPaymentClient.finish()  
}
```

12 Transaction routing

The library chooses a route per transaction, without any parameter from you.



Routing is derived from which `UserData` fields you populate. On the Interswitch path the library additionally checks a partner threshold and falls back to NIBSS when the transaction exceeds it.

Choosing your route

YOU WANT	POPULATE
NIBSS only simplest; start here	Leave <code>mid</code> , <code>bankAccountNumber</code> and <code>terminalSerialNumber</code> as empty strings. Every transaction goes straight to NIBSS.
Interswitch-enabled	Populate all three, plus <code>institutionalCode</code> . <code>init()</code> then acquires an Interswitch token at configuration time and refreshes it as needed.

The NIBSS-only route has no token lifecycle and no threshold behaviour, so it is the simpler one to get working first.

13 Responses and error handling

Three distinct outcomes

Every call resolves into one of three states. Handle all three explicitly:

OUTCOME	HOW IT ARRIVES	WHAT IT MEANS
Approved	<code>onSuccess</code> , <code>responseCode == "00"</code>	Funds authorised. Print the receipt, release goods.
Declined	<code>onSuccess</code> , other <code>responseCode</code>	The host answered and said no. This is a <i>successful</i> Rx emission. Never treat it as an error path.
Failed	<code>onError</code> , or a <code>null</code> emission	No definitive host answer, or a malformed request. See below.

Common decline codes

CODE	MEANING	ACTION
00	Approved	Complete the sale
51	Insufficient funds	Ask for another card
55	Incorrect PIN	Retry; the card tracks its own PIN try counter
57	Transaction not permitted to cardholder	Wrong account type; retry with a different <code>IsoAccountType</code>
61	Exceeds withdrawal limit	Reduce the amount
91	Issuer or switch inoperative	Retry later; do not re-present blindly
96	System malfunction	Retry later
23	Unable to go online, offline declined	Retry when connectivity returns

Codes are the standard ISO 8583 field 39 set as returned by NIBSS. Always display `responseMessage` to the operator rather than the raw code.

NibssClientException

Configuration and transaction errors arrive as `NibssClientException`, which carries a diagnostic payload the library populates for you:

```
class NibssClientException(message: String?, val nibssError: NibssError?) : Exception(message)

data class NibssError(
    val code: Int?,
    val error: String?,
    var possibleCause: String?,    // pre-filled diagnosis
    var possibleSolution: String? // pre-filled remedy
)
```

Log `possibleCause` and `possibleSolution`; they map directly onto the failures below.

ERROR MESSAGE	CAUSE AND FIX
Terminal Id and/or terminalSerial absent	<code>init()</code> was never called, or <code>UserData.terminalId</code> was empty. Configure before transacting.
SSL credentials not configured	Key exchange has not completed. Run <code>init()</code> and wait for it to succeed.
Amount should not be less than 200 kobo	Amount below the floor, or a naira value passed where kobo was expected.
... out of bounds	Invalid TID, expired session, or NIBSS downtime. Force a re-configuration.
<code>parsedResponse.getField ... must not be null</code>	The TID is not provisioned on the host. Confirm with Netplus.

Connectivity failures

Socket-level failures produce descriptive messages rather than exceptions in some paths. Use the library's own classifier rather than matching strings yourself:

```
if (ConnectionErrorConstants.isConnectionError(responseMessage)) {  
    // timeout, DNS failure, unreachable port, no internet ...  
    // The transaction outcome is UNKNOWN. Do not release goods.  
}
```

UNKNOWN OUTCOMES

A timeout is not a decline. The transaction may have been authorised at the issuer. The library auto-generates a reversal for reversible transaction types, but your UI must never treat a connection error as a definitive "no". Hold the sale and reconcile by RRN.

14 Reliability: reversals and re-push

Automatic reversals

When a reversible transaction gets no host response, the library generates a reversal advice from the original data elements without any call from you. Reversible types are purchase, purchase with cashback, purchase with additional data, cash advance, pre-authorisation and its completion, deposit, transfer, prepaid and bill payment.

Backend re-push worker

`init()` enqueues a periodic `WorkManager` job that runs every **15 minutes** on a connected network. It re-sends transactions that were processed successfully at the host but not yet logged to the Netplus gateway. Consequences for your build:

- Do not strip `androidx.work` from the dependency graph.
- If you use a custom `WorkManager` configuration, initialise it before `init()` is called.

Local persistence

Transactions are written to a Room database inside your app's private storage, keyed by RRN. Reversals and re-pushes read from it. It is not a reporting store; build your own ledger from the `TransactionWithRemark` objects you receive, and reconcile against Netplus by RRN.